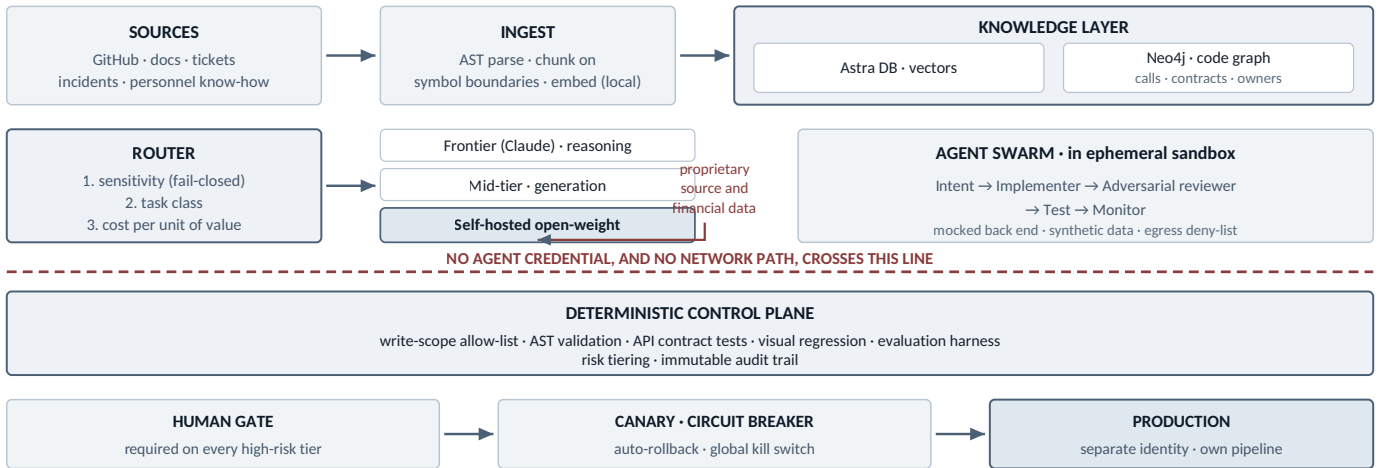


The Zevoy Collective

An architectural brief: hybrid LLM routing, a permission-aware knowledge layer, and agent swarms that cannot reach the ledger

Ujjwal Mairh · ujjwal.mairh@gmail.com · Helsinki

The premise I am designing against. Generating front-end code is the easy half and getting cheaper every month. The hard half is proving a generated change is safe before it touches a surface wired to a core banking ledger. So this brief spends most of its length on the control plane rather than the swarm, and the single architectural commitment underneath everything below is this: **no agent ever holds a production credential, and no agent output reaches production except as a reviewed artefact that passed deterministic gates.** Agents propose. The control plane disposes.



The swarm and the control plane are separated deliberately. Agents produce artefacts; a deterministic pipeline with its own identity decides what ships.

1. What I would build: the stack

1.1 Hybrid routing

One router in front of every model call, routing on three axes rather than one: task class, data sensitivity, and cost per unit of value. Sensitivity is evaluated first and is fail-closed, meaning a call that cannot be served inside its permitted boundary fails loudly rather than silently falling back to a model that is allowed less.

Task class	Tier	Why
Architecture reasoning, spec decomposition, ambiguous diffs	Commercial frontier (Claude)	Depth of reasoning dominates cost; volume is low and the output is reviewed anyway
Code generation against a known pattern, test synthesis, PR summarisation	Commercial mid-tier, open-weight where evals hold parity	High volume; routed down the moment evals show equivalence on the task's golden set
Embedding, classification, retrieval re-ranking, lint-style review, log triage	Self-hosted open-weight (Llama / Qwen class, vLLM on GPU nodes)	Very high volume, narrow task, and the input is proprietary source. Cost and IP both argue for local
Anything touching customer financial data or raw repository content	Self-hosted only, hard boundary	Data moat. Not a routing preference, a routing constraint

Routing decisions are not guesses. Every task class has a golden set, and a **parity harness** runs the cheaper tier against the expensive one before any traffic moves. Tiers move down when parity holds and move back up automatically when it degrades. I run this pattern in production today across Gemini and Vertex AI tiers with self-hosted Llama 3 underneath.

1.2 Knowledge layer: vectors plus a graph, not vectors alone

Pure vector RAG over a codebase retrieves text that looks similar, which is the wrong relation for code. An agent asked to change a payment-settings component needs to know what calls it, which API contract it depends on, who owns it, and what broke last time it changed. That is a graph question.

- Vector store (Astra DB)** for semantic recall over docs, ADRs, tickets, commit messages, incident write-ups and personnel know-how captured as structured interviews.
- Code graph (Neo4j or Astra's graph capability)** built from AST parsing, not token chunking: modules, exported symbols, call edges, route ownership, API contract dependencies, CODEOWNERS, change and incident history. Chunk boundaries follow function and component boundaries so retrieved context is always syntactically whole.
- Provenance on every node and chunk:** source, commit SHA, ingestion timestamp, sensitivity label, and the permission set that governs it. Retrieval is permission-aware by construction, not filtered after the fact.
- Continuous ingestion** on webhook from GitHub plus scheduled reconciliation, so the graph tracks reality rather than the last bulk import. Deletions and access changes propagate to embeddings, graph edges and caches in the same transaction.

1.3 Keeping the IP in

- Embeddings for proprietary source are generated by **self-hosted models**. Raw repository content never leaves the VPC for a commercial endpoint.
- Where a commercial model is genuinely needed on code, it receives **graph-resolved, minimal context**: interfaces, contracts and the specific hunk, not the repository. Smaller context is also better retrieval, so this is not purely a security tax.
- Commercial providers used only under zero-retention terms, with egress allow-lists at the network layer so the constraint is enforced by infrastructure rather than by convention.
- Every outbound call is logged with its sensitivity label, so the question "did anything proprietary ever leave" is answerable from data rather than from memory.

2. How I would secure it: the swarm and the control plane

2.1 The swarm

Specialised agents with narrow mandates beat one general agent with broad ones, because narrow mandates are auditable. Orchestration with explicit shared state. Default choice is LangGraph, for state-machine semantics and durable checkpointing, which matters because a run that dies mid-way must resume without replaying side effects. I would validate it against a bespoke state machine in a two-week spike, but the architecture does not depend on the answer and I would not block on it. Every agent action is traced and attributable.

- **Intent** agent: turns product intent or back-end state change into a scoped change proposal with explicit acceptance criteria.
- **Implementer**: produces the diff against retrieved graph context and the design system.
- **Adversarial reviewer**: a separate agent whose only job is to find reasons the diff is wrong. Different model tier from the implementer on purpose, since a model reviewing its own output is close to worthless.
- **Test**: generates and runs unit, contract and visual regression tests.
- **Monitor**: watches the deployed surface for errors, latency and conversion anomalies, and triggers rollback or escalation.

2.2 The deterministic gates

Every gate below is deterministic code, not a model judgement. A diff that fails any gate never reaches a human, let alone production.

Gate	What it enforces
Write-scope allow-list	Agents may only modify paths inside the generative UI surface. Ledger, transaction, auth and core-banking client code are outside the writable set at the filesystem and CI level. This is the first gate because it makes whole categories of failure impossible rather than unlikely.
AST validation	Diff is parsed, not string-matched. Rejects changes that introduce new network calls, new dependencies, direct fetches to banking endpoints, disabled lint or test directives, dynamic evaluation, or edits to generated client code.
API contract validation	Every call the generated UI makes is checked against the OpenAPI or GraphQL schema, with consumer-driven contract tests run against a mocked back end. A hallucinated endpoint or a wrong field type fails here, before anything deploys.
Ephemeral sandbox	Preview environments with synthetic data, no production credentials, and network egress deny-listed to everything but the mocked back end. The swarm's entire working world is this sandbox.
Test and visual regression	Unit, contract, Playwright journeys, and screenshot diffing on the golden user flows. Coverage on changed lines must hold a floor.
Evaluation harness	A golden set of representative change requests with known-good outcomes. Every prompt, model or agent-graph change is regression-tested against it before promotion. This is the mechanism that keeps the system honest as models move underneath it.
Risk tiering and HITL	Changes are scored deterministically: copy and styling on a non-transactional surface is low; anything touching a form that submits, a permission check, or a money-rendering component is high and always requires a named human approver. The tier is computed from the AST and the graph, not proposed by a model.
Canary and circuit breaker	Progressive rollout with automatic rollback on error-rate, latency or conversion regression. A global kill switch disables autonomous deployment entirely, and a repeated-failure breaker suspends the swarm pending human review rather than letting it retry into a hole.
Immutable audit trail	Every agent action, retrieved context, model and tier used, gate result and human decision recorded and replayable. This is the artefact a regulator or an auditor asks for, and it is far cheaper to build now than to reconstruct later.

The non-negotiable. The agent swarm has no credentials for, and no network path to, production banking APIs. It operates on a mocked back end behind contract tests. Promotion to production is a separate, human-gated pipeline with its own identity. The strongest guarantee against a rogue execution is not a smarter model, it is an agent that was never issued the keys.

3. When it goes live: 30, 60, 90

Day 30: the knowledge layer, read-only

- Ingestion live for GitHub, docs and tickets. Astra DB populated, code graph built from AST, provenance and sensitivity labels on every node.
- Self-hosted embedding and inference running; router in place with sensitivity routing enforced and logged.
- Retrieval quality measured against a hand-built question set, not vibes.
- **Shipped and useful already:** a retrieval assistant over Zevoy's institutional knowledge. No code generation yet. This exists partly to earn trust and partly because if retrieval is poor, generation built on it is worse.

Day 60: generation in the sandbox, human in the loop

- Write-scope allow-list, AST validation, contract testing and ephemeral sandboxes all live in CI.
- Implementer and adversarial reviewer agents producing diffs on one low-risk surface. Every diff goes to a human as a normal pull request.
- Evaluation harness and golden change set running; risk tiering computed and displayed on each PR.
- **Honest measure at Day 60:** what proportion of agent PRs merge without human edits, and what the review load looks like. If that number is poor, Day 90 moves. I would rather tell you that in week eight than in week thirteen.

Day 90: first autonomous deployment, narrow and guarded

- One well-chosen low-risk surface deploying autonomously: a non-transactional, read-only area of the app, fully behind canary, rollback and the kill switch.
- Monitor agent live on that surface, with the circuit breaker wired to real signals.
- Full audit trail queryable end to end.
- Everything else still human-gated and shipping that way.

What is honestly not done at Day 90

- Autonomy anywhere near transactional surfaces, forms that submit, or anything rendering balances. That is a Day 180-plus conversation and only after the audit trail has months of evidence behind it.
- Full swarm autonomy across the whole front end. Day 90 is one surface proven, not the app.
- Personnel know-how ingestion beyond what has been deliberately captured. Tacit knowledge does not vectorise itself, and pretending otherwise is how these projects lose credibility in month four.

4. What it costs, and why the routing pays for itself

The economic case for the hybrid design is volume distribution, not model price lists. Call volume is heavily skewed toward cheap work: embedding, re-ranking, classification and lint-style review are the overwhelming majority of calls and the least demanding. Frontier reasoning is a small fraction of calls and is reviewed by a human anyway.

Layer	Share of calls	Cost behaviour
Self-hosted open-weight	~80%	Fixed GPU line item rather than per-token. Marginal cost approaches zero as volume grows, which is the opposite of the commercial curve. This is where the swarm's inner loop lives.
Commercial mid-tier	~15%	Per-token, but routed down to local the moment the parity harness shows equivalence on that task's golden set. Expect this share to shrink over the first two quarters.
Frontier	~5%	Per-token and expensive, but low volume by construction. Capped per run, with spend attributed per agent and per change.

I would not quote you a monthly figure before seeing real volumes, because the honest answer depends on how many change requests the swarm actually handles. What I would commit to is the instrumentation: cost attributed per agent, per change and per surface from Day 30, so the routing decisions are made against measured spend rather than intuition. That metering is the same mechanism I use today to expose cost-to-serve per customer account.

5. Assumptions and the questions I would ask first

- What is the actual boundary of the "front-end layer" the swarm may own, and which surfaces are transactional? The guardrail design is only as good as that line, and I would want it drawn with your CPO in week one.
- Is there GPU capacity, or a budget for it, for self-hosted inference? The IP containment argument depends on it.
- What regulatory regime applies to change control on customer-facing banking surfaces in your licences? That determines how much of the audit trail is nice-to-have versus mandatory.
- Astra DB is specified and is a reasonable choice. I have used other vector stores; the architecture above does not depend on which one, and I would not relitigate a decision you have already made without a strong reason.